Лабораторна робота №6

Тема: Quartz Scheduler

Як користувач цієї програми, я хочу, щоб регулярні завдання (очищення, нагадування, формування звітів) виконувалися за розкладом, який зберігається після перезапуску; це гарантує, що розгортання нової версії не призведе до непомітного скасування або дублювання цих завдань.

Опис:

У межах власного проєкту визначити рутинні задачі, які повинні виконуватися або з певною періодичністю або за конкретно визначених умов (надсилання листа-підтвердження після створення нового замовлення).

Реалізація:

1. Знайти схему таблиць quartz-scheduler для власної бази даних.
2. Додати створення та менеджмент схеми Quartz через Liquibase-подібні системи.
3. Налаштувати Quartz jdbc job store.
4. Реалізувати два завдання (jobs). Два класи завдань (Job) із компонентами (beans) JobDetail і Trigger в одному класі конфігурації (з анотацією @Configuration) Quartz. Принаймні одне із завдань (Job) повинно мати два тригери.
5. Додати передачу параметрів через JobDataMap. Завдання (Job) містить значення за замовчуванням; кожен тригер може перевизначати деякі з них.

6. Безпечний повторний запуск і стійкість до збоїв. Логіка завдання має бути ідемпотентною (повторний запуск одразу після першого не змінює результат).
7. У разі збою застосовується чітко визначена стратегія: або фіксація помилки з очікуванням наступного запланованого запуску, або повторний запуск (з обмеженням кількості спроб).

Сценарій: Завдання (Jobs) та тригери оголошуються в одному місці

GIVEN клас із анотацією @Configuration, що містить біни JobDetail та Trigger

WHEN застосунок запускається

THEN кожен тригер вказує на відповідне завдання за допомогою JobKey

AND кожне завдання та тригер мають чітко визначені назву (name) і групу (group)

Сценарій: Параметри тригера мають пріоритет над значеннями завдання за замовчуванням

GIVEN опис завдання (JobDetail), що містить значення за замовчуванням у JobDataMap

AND тригер, який містить інше значення для одного з ключів

WHEN ці два набори даних (JobDataMap) об’єднуються так, як це робить Quartz

THEN значення з тригера має пріоритет

AND ключі, які не визначені в тригері, зберігають значення за замовчуванням, задані в завданні (Job)

Сценарій: Завдання (Jobs) є «легкими» (мінімалістичними)

GIVEN клас завдання

WHEN воно виконується

THEN воно зчитує свої параметри та здійснює один виклик сервісу Spring

AND не містить власної бізнес-логіки

AND межа транзакції проходить на рівні методу сервісу

Сценарій: У разі невідповідності параметрів завдання виникає явна помилка

GIVEN завдання, яке зчитує значення зі свого JobDataMap

WHEN ключ вказано з помилкою або він відсутній

THEN тест демонструє результат (викидається виняток або використовується задокументоване значення за замовчуванням)

Сценарій: Паралельне виконання заборонено, якщо воно може зашкодити

GIVEN завдання, яке не повинно обробляти одні й ті самі дані двічі одночасно

WHEN під час настання часу наступного запуску попередній запуск ще триває

THEN наступний запуск очікує завершення попереднього (@DisallowConcurrentExecution)

Сценарій: Кожен тригер має власну визначену поведінку на випадок пропуску запуску (misfire)

GIVEN тригер

WHEN він створюється

THEN для нього чітко визначено інструкцію на випадок пропуску запуску (misfire instruction)

AND тест фіксує цю інструкцію

Сценарій: Cron-тригери не залежать від часового поясу сервера

GIVEN Cron-тригер

WHEN він створюється

THEN для нього чітко визначено часовий пояс

AND тест перевіряє ідентифікатор часового поясу (невідомий ідентифікатор автоматично замінюється на GMT)

Сценарій: Повторний запуск завдання є безпечним

GIVEN сервіс, що виконує логіку завдання

WHEN логіка завдання виконується двічі поспіль для одних і тих самих даних

THEN другий запуск нічого не змінює

AND це підтверджується тестом it

Сценарій: Невдале виконання обробляється цілеспрямовано

GIVEN компонент, який викликає виняток під час виконання

WHEN завдання виконується

THEN помилка фіксується, або виконання запускається повторно (з обмеженням кількості спроб)

AND не виникає нескінченного циклу

AND тест демонструє обрану поведінку

Сценарій: Розклад зберігається в базі даних за допомогою Liquibase

GIVEN сховище завдань на основі JDBC

WHEN застосунок запускається

THEN таблиці QRTZ_* існують завдяки changeset Liquibase

AND Spring Boot налаштовано не створювати їх (параметр `initialize-schema: never`)